

01/15/14



Ceci n'est pas une glitch by the author (2010)

Glitch & Human/Computer Interaction

Daniel Temkin, 2014

Glitch art mythologizes the computer error as its ultimate muse and most potent tool: the event that triggers each piece to manifest. The glitch aesthetic may be rooted in the look of malfunction, but when it comes to actual practice, there's often not much glitch in glitch art. Yes, some glitch artists are actually exploiting bugs to get their results – but for most it would be more accurate to describe these methods as introducing noisy data to functional algorithms or applying these algorithms in unconventional ways. But this is also not algorithmic art as we ordinarily think of it – it is a more demented form of generative art: zombie algorithms stripped of their ordinary purpose or built with no clear purpose at all, set loose to twist data into strange new patterns, or to expose side-effects unwanted by their original designers. Glitch art, when approached this way, becomes a study of the dialogue between us and the machine – how we relate to logical systems, and what happens in the breakdown between human thought and computer logic. This paper will focus on this algorithmic strain of glitch art. We will look at glitch without the glitch.



from Postcards From Google Earth by Clement Valla (2011)

“...looking closer, I realized the situation was actually more interesting — these images are not glitches. They are the absolute logical result of the system.” - Clement Valla^[1]

Glitch as an Algorithmic Art

In “Notes on Glitch,” Hugh Manon and I write that “it is a given program’s failure to fully fail upon encountering bad data that allows a glitch to appear.” But let’s take a closer look at what bad data is, and in what sense the program fails (or fails to fail). As

FEATURES v1.2:

Jon Cates - POST GLITCH

Daniel Temkin/Pall Thayer - In Conversation

Laura O'Reilly - #WALLPLAY (interview)

Jeremy Bailey (interview)

Rollin Leonard/Lorna Mills - In Conversation

Kim Cascone - Errormancy

Anthony Antonellis - Internet Art in a Gallery

RSH - Cypypasta & the 8Bit Romantics

FEATURES v1.1:

Ed Winkleman - MOVING IMAGE (interview)

La Turbo Avedon (interview)

Michael Borrás, aka Systaime (interview)

Annette Doms - UNPAINTED (interview)

Daniel Temkin - Glitch & Human/Computer Interaction

Raymond Salvatore Harmon - Art 2.0

Jon Cates - kNOOwww.ART

Kelani Nichole - TRANSFER (interview)

Sean Capone - Screen Savers

RSH - The Map is More than the Territory

Daniel Rourke - LOOPS (curated gifs)

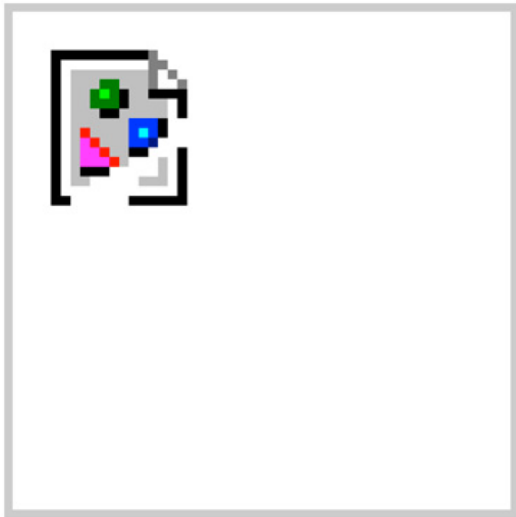
Miroslaw Rogala - The Virtual and the Vivid

an example, I'm going to use one of the foundational glitch methods, that of JPEG "corruption." But datamoshing, the WordPad effect, or many other common techniques would serve equally well.

Ted Davis's FFD8 project^[2] loads random Creative Commons-licensed images from Flickr and exposes their underlying data (displayed in raw hex). It allows us to select bytes of data (a byte here is two adjacent hex digits) and search across the field of the image to replace all instances of that byte with another. For example, we can replace all the "DF"s with "1A"s. The result is an image which becomes glitchy in appearance as the image data is altered.

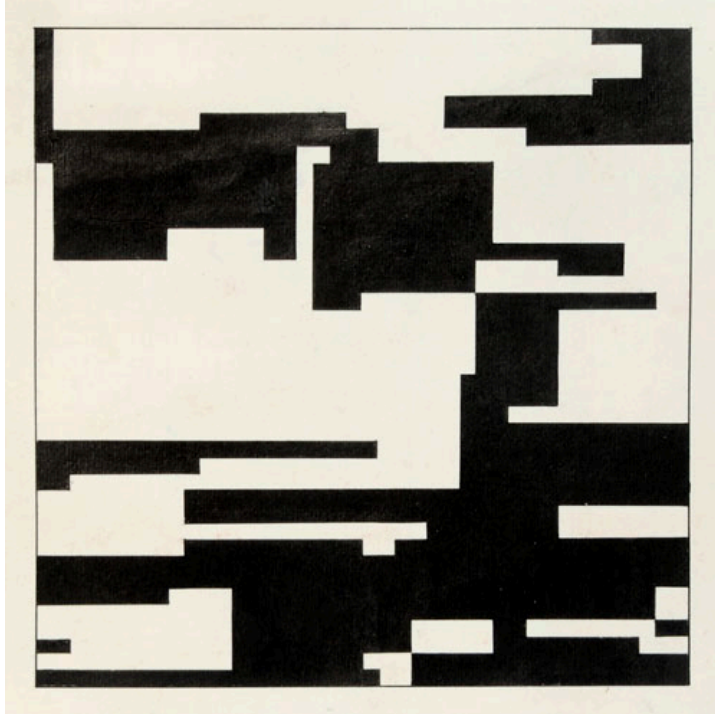
The exposure of the code-behind-the-image is part of what helps us recognize this as glitch art. The raw data is exposed for us to hack – this is how many of us began in glitch art, messing with an image data directly in a hex editor, working blindly or referencing glitch tutorials or ancient white papers for file formats. This "broken JPEG" look is one of the most recognizable of glitch aesthetics. It's familiar from Ben Syverson's Satromizer, which also alters JPEG data on the byte level, corresponding to the place where one touches the image. When we alter JPEG data this way, changes cascade through a region of the image.

JPEG is not just a file format, but an algorithm to compress and decompress image data (actually, the file format is called JPEG JFIF). What gives JPEG corruption its signature look is the way that data for each pixel is not mapped one-to-one to a place in memory but distributed within a matrix, along with the changes we introduce (the "error"). We have not actually "broken" the image in any meaningful sense; we've introduced no structural damage.



What images that succeed to fully fail look like

To go back to the question of "bad data" and "failure" of the program, to the machine, good or bad data in this context is the same; it does not care whether an image is representational or made abstract through a form of data manipulation. If we still see an image, the JPEG algorithm has successfully rendered it. Even if we somehow make the file undisplayable (which FFD8 protects us from by hiding data from before the FFD8 marker, the beginning of pixel data), we risk no failure – there's nothing at risk when digital files are effortless to duplicate and store. The data is "bad" only to us, and only if we have an expectation of representational imagery. Rather than breaking the JPEG algorithm, we've repurposed it to manufacture an aesthetic, something noisier and more chaotic, a "Dirty New Media,"^[3] revealing the digital character of the image and the patterns of JPEG compression. Glitch art underscores the computer as an apparatus indifferent to the readability or quality of the resulting image. The tension in the form does not come from risk of damage or failure, but from the surrender of the image to an unpredictable system, the collaboration with the machine.



Hiroshi Kawano, Design 2-1, Markov Chain Pattern (1964)

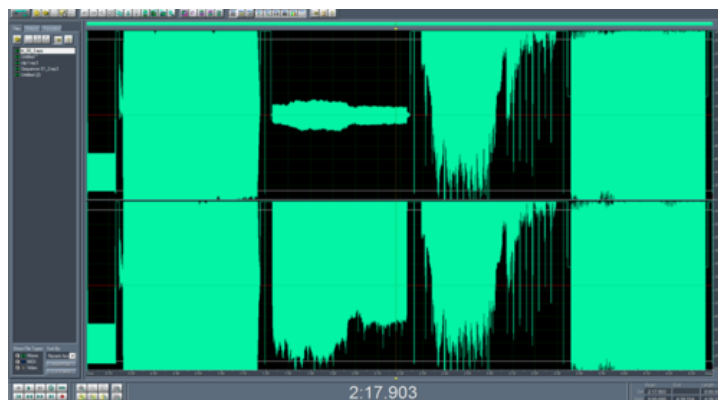


Jeff Donaldson, panasonic wj-mx12 video feedback (2012)

Kawano uses Markov Chains, a mathematical system, to generate patterns. Jeff Donaldson takes a video mixer and plugs output into input, amplifying and distorting noise through feedback

The Wilderness in the Machine

Another common glitch technique, sonification, makes this tension more clear by plying algorithms against each other. Sonification is the transformation of visual data using audio tools; one opens an image in an audio editor and adds sound effects, distorting and breaking down the image. We talk a lot about entropy in glitch art, but if entropy fully described what how audio effects transformed an image, then continuing to apply these effects would make the image progressively chaotic, until it collapsed into a featureless grey fog.



Sonification: working in the sound editor, we can't immediately see image, but the shape of the waveform gives clues to how our changes affect it

Instead, as we continue to apply effects, new forms take shape in the images, ones which have little to do with the original image. These patterns arise from the logic of sound. Delay effects tend to bend shapes, longer or shorter depending on their period, and panning left or right can create pixel-wide stripes where the visuals are offset. For sonification to work, we take advantage of the way data can be processed without the metadata which informs the machine how to interpret it, like reshaping an English sentence according to Danish rules. Each audio effect is a tool – as Curt Cloninger describes it, “a very blunt brush with a mind of its own.” This uncertainty adds much of what is exciting about making glitchwork – we give up control, not to chance, but to a system with its own patterns.

In Rosa Menkman's 2010 blog post “glitches vs glitch art”^[4] (originally from her glitch manifesto), she says “To design a glitch means to domesticate it. When the glitch becomes domesticated, controlled by a tool, or technology (a human craft) it has lost its enchantment and has become predictable.” I see this distinction as less about invoked vs found glitches (as the word “domesticated glitch” has sometimes been used), and more about employing a wildly glitchy process which retains an untamed spirit vs. one controllable to the point that it functions more-or-less as a Photoshop filter. To let the brush have a mind of its own, we use systems we don't completely understand or which are too complex to predict, allowing us to share control of the project with the machine itself. This is the glitch equivalent of generative art's emergence: where complexity reaches the level that “more comes out than the artist put in.”^[5]

Machines are, for us, the place where we most directly confront logical systems. Conceptual artists like Sol LeWitt have shown that handing off control to even a logical system of a few simple rules can take us rapidly to irrational places. As LeWitt says, “the insanity of the system is a key human condition.” Computers are constant reminders of just how bad human beings are at logic and how bizarre logic itself can be. This clash becomes apparent when we look at the place of the most direct translation between human thinking and computer logic – the writing of code.

Esoteric programming languages (esolangs) are an art form developed by programmers – programming languages which “shift attention from command and control toward cultural expression and refusal.”^[6] Brainfuck, the most notorious of these languages, destabilizes the authority of the machine by making computer logic available to the programmer in a very straightforward way, but refusing to ease the boundary between human expression and assembly code and thereby taking us on a ludicrous journey of logic. Brainfuck is a minimalist programming language, offering only eight characters recognizable by the compiler. Instead of using words like “print” to command the machine, one programs using only punctuation. The simplicity of brainfuck makes it very complex to use. There is no symbol for the number 32; instead the programmer needs to move back and forth through memory cell by cell, find a place to store the number and then increment or decrement that memory space, using a series of plus and minus signs, until 32 is achieved. Suddenly, the number 32 is no longer a constant but a resource requiring a programmatic strategy to achieve. 32 can be created by looping the addition of 4, 8 times – or by counting down (each byte has a maximum of 256) until it passes zero, perhaps several times, until landing at the correct value. Writing the number 32 suddenly allows programmers stylistic flourish.

Another take on tapping the madness of the machine is materialist exploration of data. Tapping into an algorithmic unconscious, the latent patterns in data we don't understand, has been the focus of such materialist-minded work, including Cory Arcangel's Data Diaries^[7]. The work performs as a data archeology, one which illustrates but does not explicate the content, simply allowing manic patterns to emerge. Cory describes it as “watching your computer suffocate and yell at the same time.”^[8] Similarly, LoVid's 486 Shorts^[9] literally shorts out a 486 video card, allowing super-saturated computer hallucinations to emerge. These recall works like Kurt Schwitters's Ursonate, the highly structured sound work whose literal meaning is veiled, but whose haunting patterns of alien logic come through.

Finally, Paul Hertz's GlitchSort^[10] and Kim Asendorf's PixelSort^[11] use sorting algorithms – which many of us remember as banal programming exercises from school – to create visually compelling output, rearranging pixels. These competing Processing scripts both use the captured pixels as a resource and attempt to order them, in exactly the compulsive way machines allow us to sort things. However, because these are invariably interrupted at a state halfway through, the visual effect either has a sense of a melting of objects (in Asendorf's piece) or a pattern which exposes the strangeness of each sorting algorithm (in Hertz's).

Both scripts underscore the compulsive rigor of the machine and the alien logic of its process. How visually glitchy the results look depends much on the initial image and is not necessarily important to the piece. However, as new work is built which moves the exploration of the wilderness in the machine or the compulsiveness of human logic into new visual territory, perhaps the glitch itself will become less important as a visual clue that it builds on this history of experiments in human/machine interaction.

[1] Clement Valla, “The Universal Texture,” Rhizome (2012)
<http://rhizome.org/editorial/2012/jul/31/universal-texture/>

[2] Ted Davis, FFD8, <http://ffd8.org/>

[3] <http://glitch.us/>

[4] Rosa Menkman, “glitches vs glitch art,” 4/19/2010, <http://rosa-menkman.blogspot.com/2010/04/glitches-vs-glitch-art.html>

[5] Gordon Monro, “The Concept of Emergence in Generative Art”
http://www.gommog.com/archive/docs/MMus_Essay.pdf

[6] Geoff Cox, “Speaking Code,” p.5, MIT Press, © 2013

[7] Cory Arcangel, “Data Diaries,” <http://www.turbulence.org/Works/arcangel/>

[8] Alex Galloway, Turbulence.org,
<http://www.turbulence.org/Works/arcangel/alex.php>

[9] LoVid, 486 Shorts, <http://www.lovid.org/works/486/>

[10] Paul Hertz, GlitchSort, <http://paulhertz.net/factory/2012/08/glitchsort2/>

 glitchritch **reblogged this from nooartorg and added:**
particular to note is the differentiation the author makes on “the domesticated glitch” vs original glitch. and that in...

 kattjarvis1 **liked this**

 cmpmajorproject **reblogged this from nooartorg and added:**
“glitch art it is a more demented form of generative art: zombie algorithms stripped of their ordinary purpose or built...

 photoniclabs **liked this**

 jestyrell **liked this**

 yanruchencofa-blog **reblogged this from nooartorg**

 virtualsurface **liked this**

 sinquenza **liked this**

 alikes **liked this**

 cagedrat **liked this**

 asparagusgazebo **reblogged this from nooartorg**

 asparagusgazebo **liked this**

 sartoriuskumar **liked this**

 emmablackledge **liked this**

 nooartorg **posted this**

16 notes



[←Previous](#)

[Next→](#)